Tamagui

UI Cross-platform para React Native e Web

1. O que é o Tamagui?

Sistema de UI

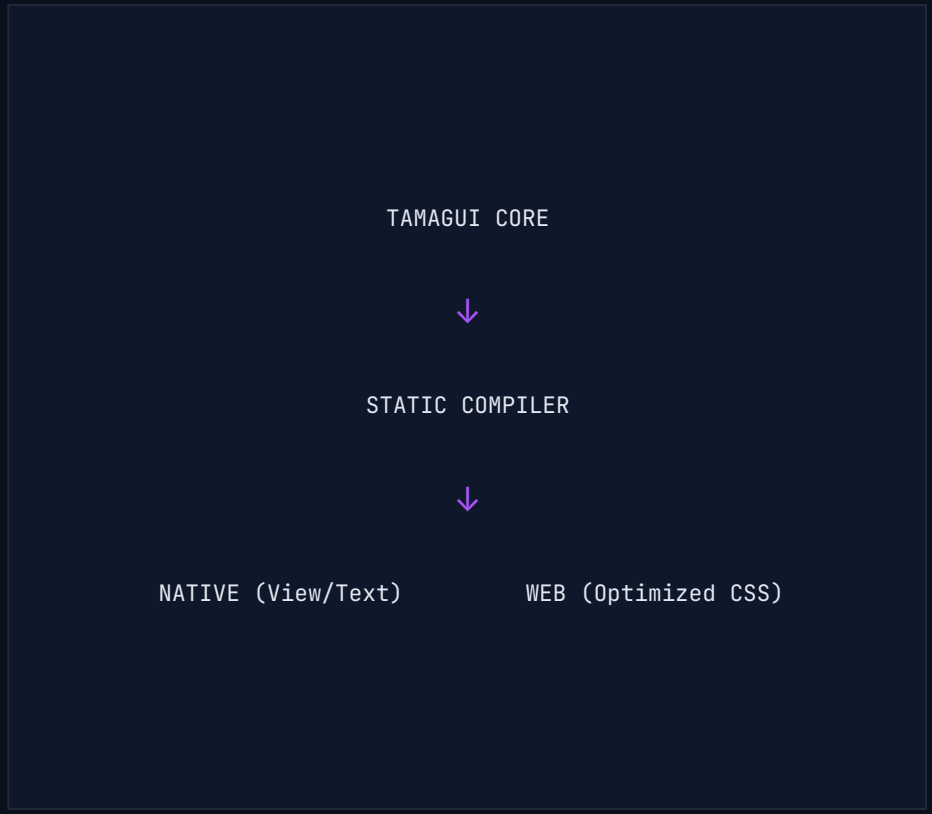
Uma suíte completa para construir interfaces que funcionam nativamente no iOS, Android e na Web.

Três Pilares Centrais

Combina um sistema de estilo (Core), um kit de componentes e um compilador otimizador.

Foco em Performance

Resolve o problema de lentidão de bibliotecas de estilo em tempo de execução através da extração estática.



2. Por que usar o Tamagui?

Produtividade

Escreva uma única vez a lógica visual e compartilhe entre o app mobile e o site sem adaptações manuais.

Performance

Compilador inteligente que converte props dinâmicas em CSS atômico altamente otimizado para a Web.

Tipagem Estrita

Suporte total ao TypeScript para estilos, tokens e temas, evitando erros comuns de runtime.

Design System Próprio

Facilita a criação de uma linguagem visual consistente com tokens de espaçamento, cores e tamanhos.

3. Como essa biblioteca funciona?

01

Definição

O desenvolvedor usa componentes do Tamagui e props de estilo no padrão React Native.

02

Compilação

O plugin de compilação (Babel/Vite) analisa o código em tempo de build.

03

Extração

Os estilos são analisados e então o Tamagui compiler observa os tokens, algumas variantes e otimiza o código.

04

Achatamento

A árvore de renderização é "achatada" (flattening), removendo o que for desnecessário.

Tudo que é dinâmico no código ainda é executado na runtime e o que é estático é executado durante a build.

4. Usos básicos

01 **Stack (XStack / YStack)**

Substitutos para o View, baseados em Flexbox horizontal ou vertical.

02 **Text & Heading**

Componentes otimizados para tipografia com suporte a tokens.

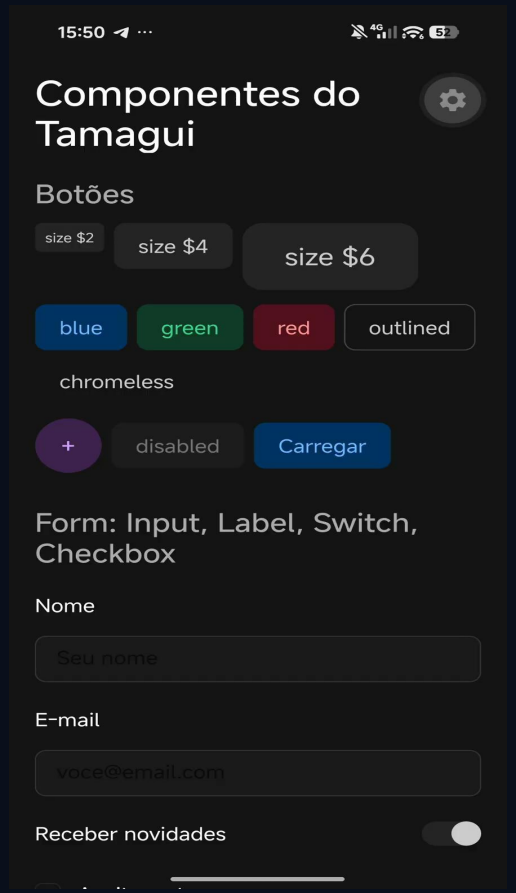
03 **styled()**

Função utilitária para criar componentes customizados com estilos padrão.

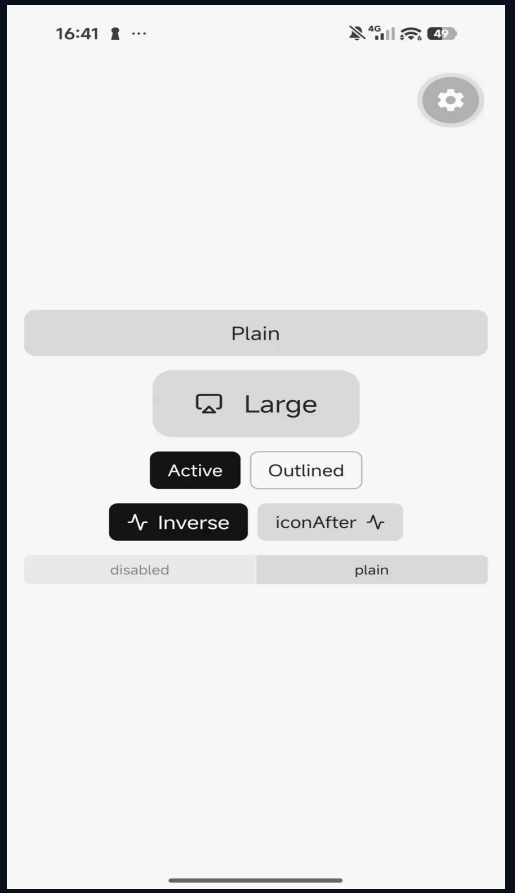
```
// Exemplo de uso
import { YStack, Text, styled } from 'tamagui'

const Container = styled(YStack, {
padding: '$4',
backgroundColor: '$background'
})

export const App = () => (
<Container>
<Text color="$color">Olá Tamagui!</Text>
</Container>
)
```


15:50
Componentes do Tamagui
Botões
size $2
size $4
size $6
blue
green
red
outlined
chromeless
+
disabled
Carregar
Form: Input, Label, Switch, Checkbox
Nome
Seu nome
E-mail
voce@email.com
Receber novidades


16:27
Modo escuro


16:41
Plain
Large
Active
Outlined
Inverse
iconAfter
disabled
plain

5. Design System + Tokens

Tokens são as variáveis fundamentais que definem a identidade visual. Eles garantem que valores como cores e espaçamentos sejam reutilizáveis e consistentes em todo o projeto.

Categoria	Token	Descrição
Cores	`$blue10`	Paleta de cores nomeada e escalonada.
Espaço	`$4`	Escala de margens e preenchimentos.
Raio	`$2`	Definição de arredondamento de bordas.
Z-Index	`$top`	Camadas de empilhamento visual.

5. Design System + Tokens

Tokens são as variáveis fundamentais que definem a identidade visual. Eles garantem que valores como cores e espaçamentos sejam reutilizáveis e consistentes em todo o projeto.

```
const tokens = createTokens({
  icon: {
    small: 10,
  },
})
```

```
export default () => (
  <Stack
    width="$icon.small"
  />
)
```

6. Variantes

Permitem criar múltiplos estados ou estilos para um mesmo componente através de props.

Facilitam a criação de botões **primários, secundários** ou tamanhos **pequeno, médio e grande**.

As variantes são **tipadas automaticamente** pelo TypeScript.

```
const MyButton = styled Button, {
variants: {
size: {
small: { padding: '$2', fontSize: 12 },
large: { padding: '$4', fontSize: 20 }
},
active: {
true: { backgroundColor: '$blue10' }
}
} as const
})

return <MyButton size "large" active />
```

7. Themes

Light Theme

$background: #FFF

$color: #000

Dark Theme

$background: #000

$color: #FFF

Nesting: <Theme name="dark"> ... </Theme>

Os temas permitem mudar o visual de partes específicas da interface dinamicamente. O Tamagui resolve os valores buscando primeiro no tema ativo e depois nos Tokens.

8. Responsividade

$gtSm (Greater than Small)
Estilos aplicados apenas em telas maiores que o breakpoint 'sm'.

$platform-web
Estilos específicos para a versão Web do aplicativo.

$platform-native
Estilos exclusivos para dispositivos móveis (iOS/Android).

```
<Stack
width={100}
backgroundColor="red"
$gtSm={{ width: 300, backgroundColor: 'blue' }}
$platform-web={{ cursor: 'pointer' }}
/>
```

9. Outros recursos

Animações

Suporte nativo para Moti e Reanimated, permitindo transições fluidas e estados de entrada/saída.

Pseudo-estados

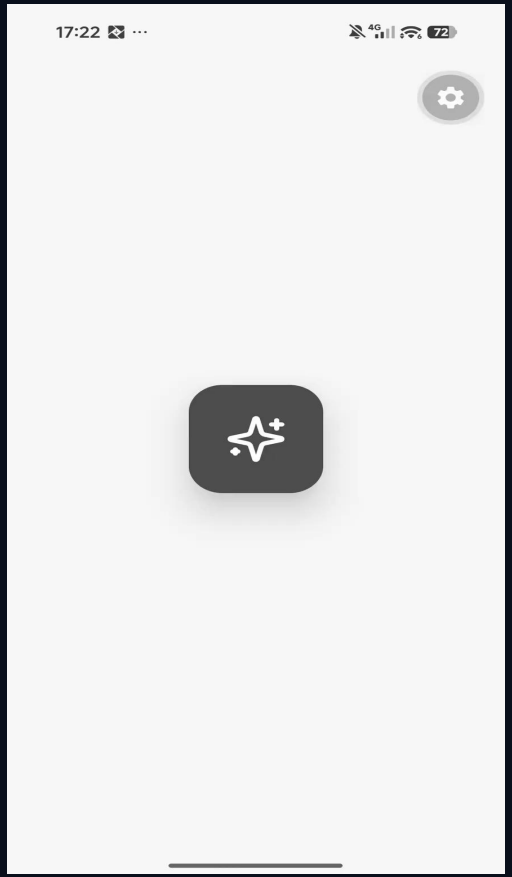
Estilização simplificada para :hover, :press e :focus de forma cross-platform.

Fontes Tipadas

Sistema de fontes com suporte a vertical rhythm e múltiplos tamanhos pré-configurados.

Depuração

Programas de depuração e props de inspeção para entender como o compilador otimizou seus estilos.


17:22

10. Vantagens e limitações

Vantagens

- Performance superior na Web (CSS estático).
- API unificada e consistente.
- Sistema de temas ROBUSTO.
- Fácil compartilhamento de código entre plataformas.

Limitações

- Curva de aprendizado inicial moderada.
- Configuração inicial mais complexa que bibliotecas simples.
- Dependência do compilador para performance máxima.
- Comunidade menor que Native Wind ou Styled Components.

11. Conclusão

- Tamagui é uma das melhores opções para UI
- Performance e Tipagem como prioridades.
- Um Design System que escala com seu app.
- Comece simples com o @tamagui/core.